



LAB PRACTICE № 9: BINARY SEARCH TREES (BST) AND ALGORITHM FOR TREE TRAVERSAL

COMP1020: OOP, Algorithms, and Data Structures

Week 09

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your classes and methods should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables, methods and classes should have meaningful names, and should be followed Java programming convention.
- Academic honesty is required in all work you submit to be graded. Although you can discuss among yourself at high level how to solve the lab problem, you should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions.
- You should upload your .java file(s) to both the Canvas and CMS Autograder **before the end of the laboratory session** unless the instructor gave a specified deadline.
 - (a) No Canvas submission will get 0 point (of course).
 - (b) **No CMS Autograder submission will get 20% deduction.**
- Submit separate *.java file for each Lab problem with the exact file name specified in each lab question (do not add your student ID or your name). For example: **HelloWorld.java**.
- Late submission of lab practice without an approved extension will incur the following penalties:
 - (a) No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 - (b) The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 - (c) The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.

Learning Outcome

Topics: Binary Search Trees (BST) and algorithm for tree traversal.

Outcome: Acquire skills of using and creating search and balanced trees. Be able to create algorithms for tree traversal in Java.

CMS Autograder

Direct access to this CMS lab is followed: <https://www.cmsvinuni.online>

Note: Due to the current limitation of the Autograder, you must comment out all the `package` declaration before submission.

Problem: Store and Search Strings

A database contains a sequence of keys $k_1, k_2, \dots, k_n$ which are strings whose lengths are from 1 to 50 ($1 \leq n \leq 100000$). Perform a sequence of actions of two kinds:

- find k : find and return 1 if k exists in the database, and return 0, otherwise
- insert k : insert a key k into the database and return 1 if the insertion is successful (k does not exist in the database) and return 0 if the insertion is failed (k exists in the database)

Build a Binary Search Tree to store the keys and provide efficient search operations.

Input

- Two blocks of information. The first block contains a key of ($k_1, k_2, \dots, k_n$) in each line. The first block is terminated with a line containing *. The second block is a sequence of actions of two finds described above: each line contains 2 string: cmd and k in which cmd = find or insert and k is the key (parameter of the action). The second block is terminated with a line containing ***. Note that the number of actions can be up to 100000.

Output Each line contains the result (0 or 1) of the corresponding action.

Listing 1: Template for Problem 1.

```
import java.util.*;

class TreeNode {
    String key;
    TreeNode left, right;

    public TreeNode(String item) {
        key = item;
        left = right = null;
    }
}

class StringBST {
    private TreeNode root;
```

```

public StringBST() {
    root = null;
}

public int insert(String key) {
    if (root == null) {
        root = new TreeNode(key);
        return 1;
    } else {
        return insertRec(root, key);
    }
}

private int insertRec(TreeNode node, String key) {
    // TODO: Recursively insert the key into the BST.
    // Return 0 if the key already exists, otherwise insert and return 1.
}

public int find(String key) {
    // TODO
}

private int findRec(TreeNode node, String key) {
    // TODO: Recursively search for the key in the BST.
    // Return 1 if found, 0 otherwise.
}

}

public class StoreAndSearchStrings {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        StringBST tree = new StringBST();
        List<String> output = new ArrayList<>();

        boolean readingInitialKeys = true;

        while (sc.hasNextLine()) {
            String line = sc.nextLine().trim();
            if (line.equals("*")) {
                readingInitialKeys = false;
                continue;
            }
            if (line.equals("***")) {
                break;
            }

            if (readingInitialKeys) {
                tree.insert(line);
            } else {
                String[] parts = line.split("\\s+");

```

P02 - Testcase 2

P02 - Testcase 1

```
computer
university
school
technology
phone
*
find school
find book
insert book
find algorithm
find book
insert book
***
1
0
1
0
1
0
```

```
apple
banana
cherry
date
elderberry
fig
grape
*
find banana
find kiwi
insert mango
insert apple
insert lemon
insert banana
find mango
find apple
find lemon
***
1
0
1
0
1
0
1
1
1
1
```

Figure 1: Testcases of Problem 01

```
if (parts.length == 2) {
    String cmd = parts[0];
    String key = parts[1];
    if (cmd.equals("insert")) {
        output.add(String.valueOf(tree.insert(key)));
    } else if (cmd.equals("find")) {
        output.add(String.valueOf(tree.find(key)));
    }
}
}
}
sc.close();

for (String result : output) {
    System.out.println(result);
}
}
```

Testcases See Figure 1